The RSA Algorithm

(**R**ivest, **S**hamir and **A**dleman)

1978

Overview

- Introduction
- RSA Encryption/Decryption
- RSA Key Setup
- How RSA works?
- RSA Example
- RSA Security
- Practice Problems
- Summary

Introduction

- RSA - Rivest Shamir Adleman
- by Rivest, Shamir & Adleman of MIT in 1978
- best known & widely used public-key scheme
- based on exponentiation in a finite (Galois) field over integers modulo a prime
- uses large integers (eg. 1024 bits)
- security due to the cost of factoring large numbers
  - factorization takes $O(e^{\log n \log \log n})$ operations (hard)

RSA: Encryption and Decryption

Encryption:

- to encrypt a message M the sender:
  - obtains public key of recipient PU={e, n}
  - computes: $C = M^e \bmod n$, where $0 \leq M < n$

Decryption:

- to decrypt the ciphertext C the owner:
  - uses their private key PR={d, n}
  - computes: $M = C^d \bmod n$
- note that the message M must be smaller than the modulus n (block if needed)

RSA Key Setup

Key generation:

- each user generates a public/private key pair by:
- selecting two distinct large primes at random: p, q and kept secret
- computing their system modulus $n=p.q$
  - note $ø(n)=(p-1)(q-1)$
- selecting at random the encryption key e
  - where $1<e<ø(n)$, $gcd(e,ø(n))=1$
- solve the following equation to find the decryption key d
  - $e.d=1 \bmod ø(n)$ and $0 \leq d \leq ø(n)$
- publish their public encryption key: $PU=\{e,n\}$
- keep secret private decryption key: $PR=\{d,n\}$

How RSA works?

- works because of Euler's Theorem:
  - $a^{ø(n)} \bmod n = 1$ where $\gcd(a,n)=1$

- in RSA have:
  - $n=p.q$
  - $ø(n)=(p-1)(q-1)$
  - carefully chose e & d to be inverses mod ø(n)
  - hence $e.d=1+k.ø(n)$ for some k
- hence :

$$C^d = M^{e.d} = M^{1+k.ø(n)} = M^1.(M^{ø(n)})^k$$

$$= M^1.(1)^k = M^1 = M \bmod n$$

RSA Example – Key Setup

1. Select primes: p=17 & q=11
2. Calculate $n = pq$ =17 x 11=187
3. Calculate $ø(n)=(p–1)(q-1)$=16x10=160
4. Select e: gcd(e,160)=1; choose e=7
5. Determine d: ed=1 mod 160 and d < 160 Value is d=23 since 23 x 7=161= 10 x 160 + 1
6. Publish public key PU={7,187}
7. Keep secret private key PR={23,187}

RSA Example – Encryption and Decryption

- sample RSA encryption/decryption is:
- given message M = 88 (nb. 88<187)
- encryption:
  - $C = 88^7 \bmod 187 = 11$
- decryption:
  - $M = 11^{23} \bmod 187 = 88$

Exponentiation

- can use the Square and Multiply Algorithm
- a fast, efficient algorithm for exponentiation
- concept is based on repeatedly squaring base
- and multiplying in the ones that are needed to compute the result
- look at binary representation of exponent
- only takes $O(\log_2 n)$ multiples for number n
  - eg. $7^5 = 7^4.7^1 = 3.7 = 10 \bmod 11$
  - eg. $3^{129} = 3^{128}.3^1 = 5.3 = 4 \bmod 11$

Example 2

P=5 & q=7
n=5*7=35 and ø(n)=(4)*(6) = 24
e = 5
d = 29 , (29x5 =1 mod ø(n)=24)
Keys generated are
Public key: (e, n)=(5,35)
Private key is (d, n)=(29, 35)
Encrypt the word love using ($c = m^e$ mod n)
Assume that the alphabets are between 1 & 26

Plain Text	Numeric Representation	m^e	Cipher Text ($c = m^e$ mod n)
l	12	248832	17
o	15	759375	15
v	22	5153632	22
e	5	3125	10

Example 2 contd…

Decrypt the word love using ($m = c^d \bmod n$)

$n = 35$, $c=29$

Cipher Text	c^d	($m = m^e \bmod n$)	Plain Text
17	481968572106750915091411825223072000	17	l
15	12783403948858939111232757568359400	15	o
22	852643319086537701956194499721110000000	22	v
10	100000000000000000000000000000	10	e

Efficient Encryption

- encryption uses exponentiation to power e
- hence if e small, this will be faster
  - often choose e=65537 (2^{16}-1)
  - also see choices of e=3 or e=17
- but if e too small (eg e=3) can attack
  - using Chinese remainder theorem & 3 messages with different modulii
- if e fixed must ensure gcd(e,ø(n))=1
  - ie reject any p or q not relatively prime to e

Efficient Decryption

- decryption uses exponentiation to power d
  - this is likely large, insecure if not
- can use the Chinese Remainder Theorem (CRT) to compute mod p & q separately.
- then combine to get desired answer
  - approx 4 times faster than doing directly
- only owner of private key who knows values of p & q can use this technique

RSA Key Generation

- users of RSA must:
  - determine two primes at random - p, q
  - select either e or d and compute the other
- primes p,q must not be easily derived from modulus n=p.q
  - means must be sufficiently large
  - typically guess and use probabilistic test
- exponents e, d are inverses, so use Extended Euclidean algorithm/Inverse algorithm to compute the other

RSA Security

- possible approaches to attacking RSA are:
  - brute force key search - infeasible given size of numbers
  - mathematical attacks - based on difficulty of computing ø(n), by factoring modulus n
  - timing attacks - on running of decryption
  - chosen ciphertext attacks - given properties of RSA

Factoring Problem

- mathematical approach takes 3 forms:
  - factor n=p.q, hence compute ø(n) and then d
  - determine ø(n) directly and compute d
  - find d directly

- currently assume 1024-2048 bit RSA is secure
  - ensure p, q of similar size and matching other constraints

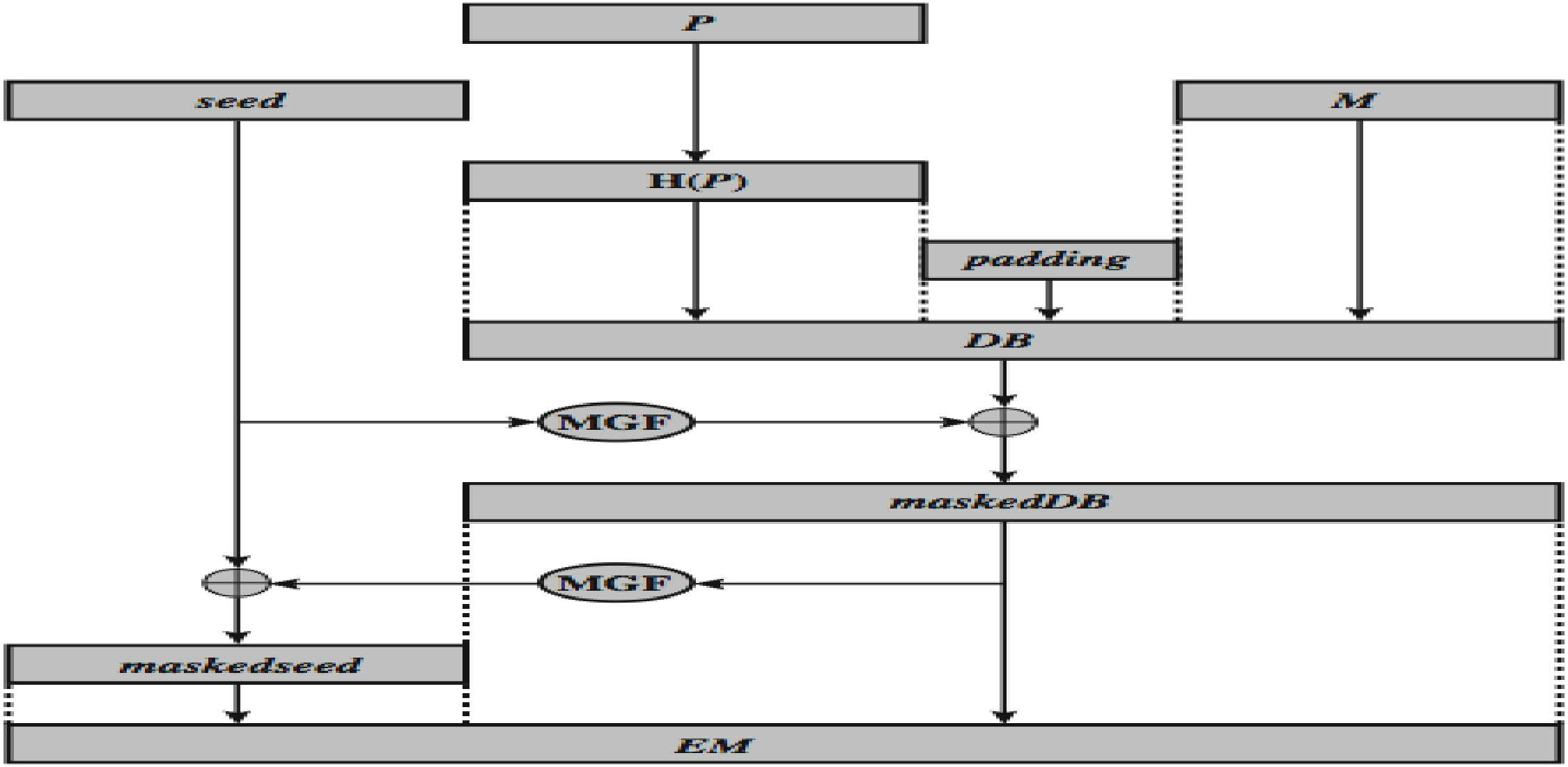
Timing Attacks

- developed by Paul Kocher in mid-1990's
- exploit timing variations in operations
  - eg. multiplying by small vs large number
  - or IF's varying which instructions executed
- infer operand size based on time taken
- RSA exploits time taken in exponentiation
- countermeasures
  - use constant exponentiation time
  - add random delays
  - blind values used in calculations

Chosen Ciphertext Attacks

- RSA is vulnerable to a Chosen Ciphertext Attack (CCA)
- attackers chooses ciphertexts & gets decrypted plaintext back
- choose ciphertext to exploit properties of RSA to provide info to help cryptanalysis
- can counter with random pad of plaintext
- or use Optimal Asymmetric Encryption Padding (OASP)

Optimal Asymmetric Encryption Padding (OASP)



P = encoding parameters
M = message to be encoded
H = hash function

DB = data block
MGF = mask generating function
EM = encoded message

Practice Problems

1) Perform encryption and decryption using the RSA algorithm:

a) p=3; q=11; e=7; M=6

b) p=11; q=13; e=11; M=7

c) p=7; q=11; e=17; M=8

2)In a public-key system using RSA, you intercept the ciphertext C=10 sent to a user whose public key is e=5, n=35. What is the plaintext M?

Summary

Discussed about:

- Introduction
- RSA Encryption/Decryption
- RSA Key Setup
- How RSA works?
- RSA Example
- RSA Security
- Practice Problems